Decomposition and Coordination for Long-Horizon Agents

Multi-Agent Patterns, Trade-offs, and Frontier Bets

Matt Redmond

1 Apr 2026

Today's Journey

Earlier in the course, the central design question was often:

How do we make one model call, or one agent loop, work better?

This unit asks a more systems-oriented question:

The through-line

When tasks get long, stateful, and messy, what should be kept in one thread, what should be delegated, and how should the pieces synchronize?

Part 1

1. Reframing the problem
2. What current evidence supports
3. Core design tensions
4. The main bottlenecks

Part 2

5. Decomposition patterns that mostly work
6. Patterns that help but add inertia
7. Frontier ideas: Slate, Manus, Gastown, Beads
8. What to claim with confidence today

Learning Objectives

By the end of this unit, you should be able to:

1. explain why "multi-agent" is only one part of the broader long-horizon agent design space
2. distinguish strong current patterns from speculative frontier bets
3. reason about decomposition using the key trade-offs: **shared context vs isolation**, **planning vs reactivity**, and **memory vs lossiness**
4. understand patterns like routers, fan-out/fan-in, verifier loops, planner–executor systems, and hierarchical delegation
5. discuss how anti-multi-agent arguments fit into the current landscape

Outline

1. Reframing the Problem
2. Patterns That Mostly Work
3. Patterns That Help but Add Inertia
4. The Central Debate: Synchronization
5. Frontier Bets

The Wrong First Question

The tempting, superficial question is:

"How do I make a **team of agents**?"

The more useful question to ask is actually:

"How do I decompose work while preserving the **right context**, **right control flow**, and **right synchronization boundaries**?"

In other words, the topic is not really "agent societies" or anything fanciful like that.

It is less sci-fi: how do you do **systems design for long-horizon language-model work?**

What the Stronger Evidence Actually Supports

Reasonably well supported

- long contexts degrade usefulness, not just cost
- reactive loops often beat rigid plans
- adaptive decomposition helps on complex tasks
- context management is central
- bounded externalized state helps

Still unsettled

- many agents vs one threaded agent
- message-passing subagents as primitive
- planner–implementer–reviewer durability
- orchestration-heavy "agent factories"

The left column has repeated empirical support. The right column is mostly product opinions.

Four Core Tensions

Most of the design space can be understood through four tradeoff axes:

Tension	Left side	Right side
Context	shared context	isolated local contexts
Control flow	reactive step-by-step loops	explicit plans and schedules
Memory	keep everything in-thread	compress / externalize / retrieve
Organization	one generalist loop	many specialists or workers

Most architectures are just different points in this trade-off space.

The Main Long-Horizon Bottlenecks

Once tasks become long-running, the main bottlenecks are usually:

1. **Context management**: too much history, too much noise, too much loss from compression
2. **Synchronization**: different parts of the system acting on conflicting assumptions
3. **Control**: too much rigidity or too little structure
4. **Evaluation**: end-to-end failure without visibility into where things broke

The key lecture claim today is this:

> The hard part is often not "more intelligence." It is **better decomposition and coordination.**

An Evidence Ladder

Not every argument in this space deserves equal confidence.

1. **Most confidence**: repeated empirical findings on context limits, reactive loops, and adaptive decomposition
2. **Moderate confidence**: practitioner reports from systems like Manus, Devin, Claude Code, Amp
3. **Lower confidence**: analogies from board games, operating system metaphors, and strong product opinions
4. **Least confidence**: sweeping claims about the inevitable future of agent teams

We'll try to separate:

- **what seems robust**
- **what seems promising**
- **what is still a long bet**

Outline

1. Reframing the Problem
2. Patterns That Mostly Work
3. Patterns That Help but Add Inertia
4. The Central Debate: Synchronization
5. Frontier Bets

Pattern 1: Router / Delegate

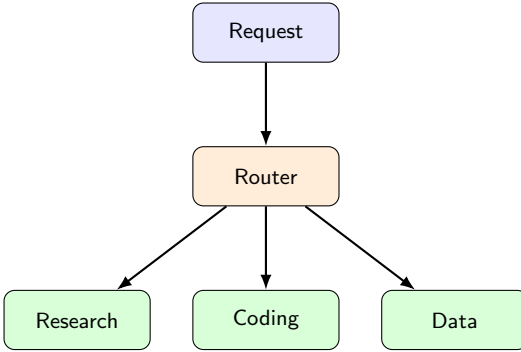
The simplest useful split on multi-agent design is often just:

1. inspect the request
2. classify the kind of work
3. delegate to the best specialist or tool cluster

This pattern is strong because it solves a narrow problem:

- specialization
- policy separation
- permission separation
- prompt simplification

Routers are not magic. They are just **classifiers with consequences**.



Router Failure Modes

Common failure modes:

- bad or overconfident routing
- hidden behavior changes when the router prompt changes
- no abstain / fallback path

```
route = classify(request)

if route.confidence < THRESHOLD:
    return fallback_agent(request)

agent = agents[route.label]
return agent.run(request)
```

Evaluate routing like any classifier: edge cases, abstention, confusion matrix.

Pattern 2: Fan-Out / Fan-In

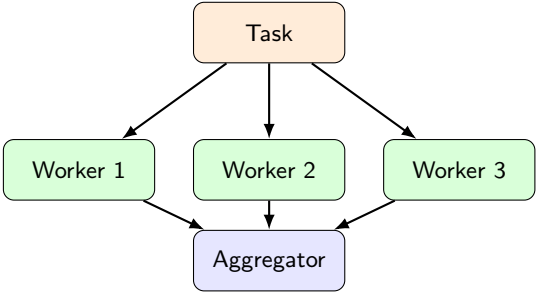
The next strong pattern is:

1. split the task into mostly independent subproblems
2. run them in parallel
3. merge the results

This is compelling when the work is naturally parallel:

- inspect many files
- summarize many documents
- classify many records
- investigate multiple hypotheses

The decomposition is often **obvious and testable**.



The Merge Is Harder Than the Split

Fan-out looks clean until the aggregator has to reconcile:

- duplicate findings
- conflicting findings
- uneven quality across workers
- uncertain results that still sound fluent

Better merge behavior usually comes from:

- structured outputs
- provenance
- explicit uncertainty
- a distinction between reducing evidence and judging conclusions

Pattern 3: Verifier Loops

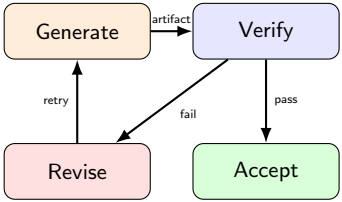
A very strong pattern is not "many agents talking."

It is:

generate → check against grounded constraints → revise if needed

This works because the verifier can often be more reliable than the generator:

- tests
- schemas
- type checks
- execution traces
- retrieval-backed evidence



Critic Versus Verifier

These are not the same thing:

Critic

- another LLM comments
- often useful
- often vague
- can share the same blind spots

Verifier

- grounded in rules or execution
- often narrower
- much easier to trust
- much easier to evaluate

In practice, verifier loops deserve much more confidence than free-form debate or critique loops.

Outline

1. Reframing the Problem
2. Patterns That Mostly Work
3. Patterns That Help but Add Inertia
4. The Central Debate: Synchronization
5. Frontier Bets

The Planning Question

There is a real reason planner-style systems keep reappearing:

- they make long tasks legible
- they force explicit decomposition
- they can reduce early stopping
- they create artifacts that can be inspected

But explicit structure introduces a different risk:

the plan can become more rigid than the environment

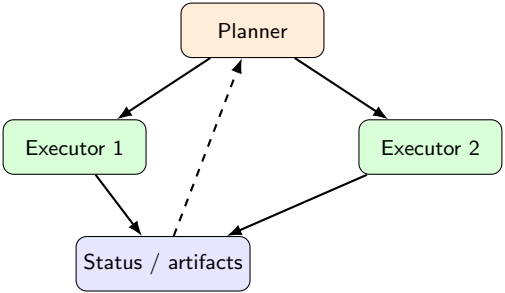
Planner–Executor: When It Helps

Planner–executor designs are strongest when:

- task order matters
- dependencies are real and visible
- explicit completion criteria matter
- the plan needs to be reviewed by a human or another system

Good examples:

- research workflows
- complex coding tasks with search/edit/test phases
- operational workflows with approvals or gates



Planner–Executor: Why It Often Feels Clunky

Well-understood failure modes:

- upfront decomposition is incomplete
- world changes during execution
- compressed status loses important state
- system follows stale plans too long

The critique

Planning helps, but too much explicit orchestration destroys flexibility.

— *Slate*, Cognition

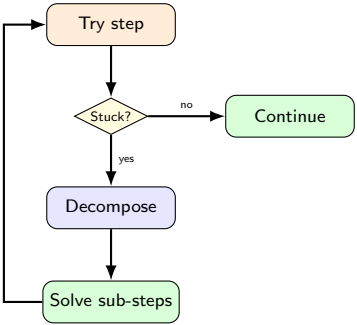
ADaPT and As-Needed Decomposition

A useful middle ground from ADaPT:

- do not decompose the task up front
- only decompose when the current step is too hard
- recurse when needed, not by default

Adaptive decomposition is usually better than rigid decomposition.

Preserves the reactivity that makes ReAct-like systems strong.



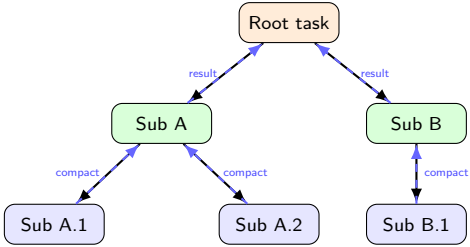
Recursive Decomposition and RLM

RLM-style approaches push the idea further:

- keep the root context small
- externalize large state
- recurse into narrower subproblems
- return compact results upward

Powerful because it combines **isolation**, **search**, and **compression**.

But it makes one problem obvious: **over-decomposition**.



Guardrails for Recursive Systems

Once recursive delegation exists, budgets become mandatory:

- maximum depth
- maximum branching
- explicit stopping criteria
- typed intermediate results
- a way to kill unproductive branches

Recursive systems are best thought of as:

a search mechanism with context isolation, not a free lunch

Outline

1. Reframing the Problem
2. Patterns That Mostly Work
3. Patterns That Help but Add Inertia
4. The Central Debate: Synchronization
5. Frontier Bets

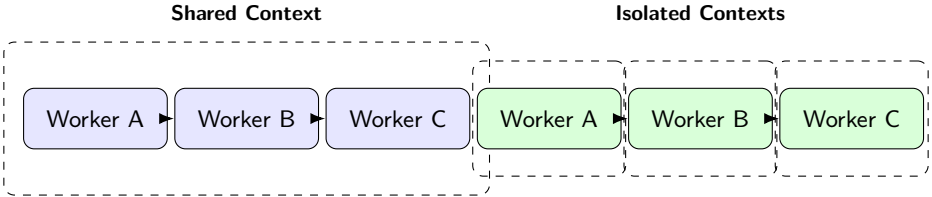
The Real Debate Is Shared Context Versus Isolation

The strongest disagreement in current agent discourse is not:

"Should we use more agents?"

It is:

How much context should be shared, and what information is safe to summarize, compress, or hand off across boundaries?



This is where the strongest product opinions diverge.

The Cognition View: Share Context

In "Don't Build Multi-Agents," Cognition argues:

- actions carry implicit decisions
- isolated agents act on different assumptions
- many parallel agents are fragile because they cannot share enough context
- single-threaded systems are often more reliable than naive agent teams

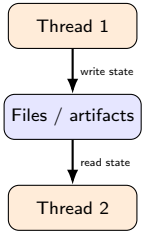
This is an important corrective:

parallelism is not automatically progress if it fragments the decision boundary

The Manus / Amp View: Externalize and Handoff Carefully

Core claims:

- context windows cannot hold everything
- files and artifacts act as external memory
- naive compaction is lossy
- handoffs must preserve what the next thread needs



The alternative to "keep everything in context" is often **externalized state**, not more agents.

Compaction, Handoff, and Episode Boundaries

Three different answers to the context problem:

Mechanism	Strength	Main risk
Compaction	keeps one thread alive	unpredictable information loss
Handoff	starts a focused new thread	depends on transfer quality
Episodes / returns	gives clean completion boundaries	still needs useful summarization

The open problem is not whether compression exists.

It is whether the **compression boundary matches the task structure**.

The Ralph Wiggum Loop

Geoffrey Huntley's "Ralph Wiggum" technique is the opposite of fancy agent society design:

> Keep one monolithic agent loop running, give it one concrete task at a time, and use repeated prompting plus backpressure to keep it on the rails.

Canonical form:

```
while :; do
  cat PROMPT.md | claude-code
done
```

The key ideas are:

- one item per loop
- stable prompt scaffolding every loop
- lots of externalized plan/spec files
- strong backpressure from tests, builds, and type checks

Why Ralph Is Interesting

Ralph is useful because it highlights an uncomfortable possibility:

> A dumb but disciplined monolithic loop may beat a clever multi-agent system if the clever system fragments context too aggressively.

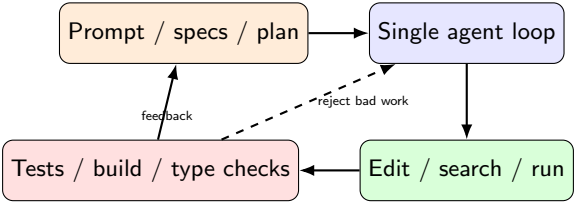
Why it can work:

- no agent-to-agent synchronization problem
- one active decision boundary
- strong local feedback from tests and build failures
- the operator can keep tuning the prompt "signs"

Why it is also absurd:

- it is basically industrialized vibe-coded Groundhog Day
- it depends heavily on operator tuning
- it can still loop, drift, duplicate work, and hallucinate state

Ralph as a Systems Diagram



The loop is primitive, but the lesson is serious:

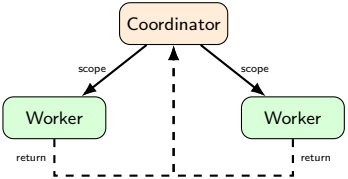
good backpressure plus one shared context can outperform more elaborate coordination schemes

The Slate View: Synchronization First

Slate makes a sharper claim:

- compaction is too lossy
- subagent message-passing isolates too aggressively
- rigid planners add too much inertia
- bounded workers + episodic returns may be better

The key contribution: putting **synchronization** at the center.



Outline

1. Reframing the Problem
2. Patterns That Mostly Work
3. Patterns That Help but Add Inertia
4. The Central Debate: Synchronization
5. Frontier Bets

Manager–Worker Systems

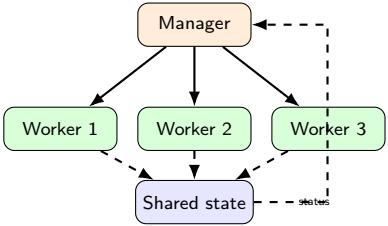
An interesting middle ground:

- more structured than fan-out/fan-in
- more dynamic than planner–executor
- supports branching, retries, scheduling

But also introduces:

- manager bottlenecks
- synchronization overhead
- stale summary accumulation

Promising, but not yet the default answer.



Hierarchies and Agent "Teams"

Full hierarchies and many-agent teams are the most speculative part of the space.

They promise:

- scale
- parallelism
- specialization
- durable division of labor

They also intensify every hard problem:

- synchronization
- stale state
- conflicting assumptions
- latency and cost
- evaluation difficulty

Gastown and Beads

Steve Yegge's *Gastown* and *Beads*: the developer becomes the foreman of an AI dockyard.

The vision

- every task is a durable work object
- agent identities persist across sessions
- orchestration is a first-class job
- scale through industrial parallelism

The concern

- elaborate workaround for brittle agents?
- your subprocesses now need paperwork
- high overhead if agents improve quickly
- unclear evaluation story

If you are willing to run a small AI shipping port, the throughput promise is real. Whether it outlasts the current generation of agents is the open question.

What These Frontier Systems Are Actually Betting On

System / view	Main bet
Cognition	shared context and single-threaded reliability beat naive multi-agent fragmentation
Manus	context engineering and external memory are central; long-horizon success comes from shaping context well
Amp	focused handoff beats meandering compaction
Slate	synchronization boundaries and episodic returns matter more than agent count
Gastown / Beads	durable orchestration and persistent task objects can industrialize coding workflows

What We Should Actually Claim Today

Reasonably confident

1. routers, fan-out/fan-in, and verifier loops are strong practical patterns
2. reactive loops are still hard to beat
3. adaptive decomposition > rigid decomposition
4. synchronization quality matters more than agent count
5. externalized state > free-form agent conversation

Be cautious about

- "the future is teams of agents"
- "planner–implementer–reviewer is the standard architecture"
- "one new primitive has clearly won"
- "shared context scales indefinitely"
- "isolated workers are fine if you just summarize well"

Worked Example: Migrate a Codebase from REST to GraphQL

Task: rewrite 40 endpoint files, update tests, keep the app passing CI.

Pattern	Verdict	Why
Single loop	viable	files are independent; strong test backpressure
Router	not helpful	one kind of work, no specialization needed
Fan-out/fan-in	**strong fit**	40 files are embarrassingly parallel; merge = concat
Verifier loop	**strong fit**	CI and type checks give grounded verification
Planner–executor	overkill	dependencies are simple; rigid plan adds inertia
Manager–worker	maybe	useful if files have cross-cutting schema changes

Likely best approach

Fan-out per file → verifier loop (CI) per worker → single merge commit. Only add a manager if cross-file schema coordination is needed.

A Practical Decision Checklist

Before adding decomposition or orchestration, ask:

1. what exact bottleneck am I solving: **quality**, **latency**, **context**, or **control**?
2. can one threaded loop solve it with better context engineering first?
3. are the subproblems truly independent?
4. what assumptions must stay shared?
5. what information can be safely compressed?
6. how will I evaluate each boundary separately?

When *Not* to Decompose

Signs you should keep a single agent loop:

- subtasks share mutable state
- the split is not obvious or testable
- pieces cannot be evaluated independently
- you have not tried better context engineering yet
- the coordination cost exceeds the parallelism gain

The anti-pattern

Premature decomposition fragments context before you understand what needs to stay together.

Rule of thumb

If you cannot write a clear interface between the parts, they are probably one part.

Discussion Questions

1. Is synchronization the core systems problem, or is it just a symptom of weak planning and weak memory?
2. Which pattern feels most useful today: routers, fan-out/fan-in, verifier loop, planner–executor, or manager–worker?
3. Are persistent task objects like Beads solving a deep systems problem, or just adding silly bureaucracy for robots?
4. If context windows became vastly better tomorrow, which of these patterns would still matter?
5. Which architectural claims that you've seen today feel like robust insights, and which feel like product-specific bets?
6. Do you think the future closer to a single very capable threaded agent, or to a managed team of narrower workers?

Closing Question

A year from now, which bet wins?

A One very capable threaded agent with great context engineering

B A managed fleet of narrow specialists with great synchronization

C Something we haven't built yet

What would you need to see to change your mind?

References and Further Reading

- Random Labs, *Slate* — strong synchronization-first argument
- Walden Yan, *Don't Build Multi-Agents* — strong skeptical view of naive multi-agent systems
- Manus, *Context Engineering for AI Agents* — practical context-engineering lessons
- Amp, *Handoff* — good discussion of compaction vs thread handoff
- Alex Zhang, *Recursive Language Models* — clear explanation of RLM ideas
- Prasad et al., *ADaPT* — as-needed decomposition and planning
- Yao et al., *ReAct* — foundational reactive agent loop paper
- Hong, Troynikov, Huber, *Context Rot* — why long contexts degrade in practice

More References

- Steve Yegge, *Welcome to Gas Town* — the agent-factory vision
- Steve Yegge, *Beads* — durable task objects and orchestration substrate
- Silver et al., *AlphaZero* — useful analogy for strategy vs tactics
- Silver et al., *AlphaGo* — another good strategy/tactics analogy
- McGrath et al., probing AlphaZero — evidence that tactical and strategic concepts emerge differently
- Geoffrey Huntley, *The Ralph Loop* — fun cautionary failure mode
- Anthropic, *Building Effective Agents* — broad practical guide
- OpenAI, *A Practical Guide to Building Agents* — another good broad overview